

Vigilância de Partições Retangulares

Projeto Prático — MAD CC3003 2025/26

Grupo 06
Cauã Pinheiro Souza (202302423)

Matheus Gonçalves Guerra (202205324)

Maio 2026

1 Introdução

O problema consiste em determinar uma colocação mínima de guardas nos vértices de uma partição retangular Π , garantindo que todos os retângulos de Π (ou um subconjunto $\Pi' \subset \Pi$ dado) ficam vigiados. Um guarda colocado num vértice p vigia todos os retângulos incidentes em p .

Notação. Seguimos a notação do problema dos “Postos de Vigia” (slides MAD2526_clp e clp_swi_2026):

- R — retângulos a cobrir.
- P — vértices candidatos a posto de guarda.
- $G[r]$ — vértices que veem o retângulo r .
- $V[p]$ — retângulos que o vértice p vê (inversa de G).
- $S \subseteq P$ — conjunto de guardas escolhidos (solução).

Pré-processamento partilhado por todas as abordagens.

1. Leitura do ficheiro produzido pelo gerador `rectParts.c` (formato: na linha 1, o número de instâncias; cada instância começa pelo número de retângulos nr , seguida de nr linhas com `id nv x1 y1 ...xnv ynv`).
2. Identificação do retângulo exterior como aquele com a *bounding box* máxima.
3. Descarte dos vértices da borda externa do candidato a guarda, pela observação da alínea (b) do exercício 7 da fp3: para qualquer solução ótima, é sempre possível trocar um guarda da borda por um vizinho interno equivalente.

2 Q1 — Estratégias *greedy*

2.1 Métodos

Avaliámos duas estratégias gulosas, ambas com origem direta no material da cadeira.

(1) *first-fail*. Em cada iteração escolher o retângulo r não coberto com menos candidatos disponíveis (menor $|G[r] \setminus S|$); entre os candidatos desse retângulo, escolher o que cobre mais retângulos ainda não cobertos. A relação com a heurística *first-fail* usada na procura de soluções de CSPs é discutida explicitamente na alínea (a) do exercício 7 da fp3 e nos slides MAD2526_clp: atacar primeiro a variável mais restringida (com menor domínio) é onde a procura é mais provável falhar e portanto onde mais se corta o espaço de procura. No nosso problema, o “domínio” de cobertura de cada retângulo é o conjunto $G[r]$ dos vértices que o veem.

(2) **max-cobertura.** Em cada iteração escolher o vértice p que maximiza $|V[p] \cap \text{nao_cobertos}|$. É a estratégia clássica para o problema de *minimum set cover* e segue o mesmo padrão das estratégias gulosas apresentadas nos slides (Kruskal: escolher o ramo de menor peso; *knapsack* fraccionário: escolher pelo maior rácio valor/peso) — escolha baseada num critério local de maior benefício por passo.

Complexidade. Ambas correm em $O(|R| \cdot |P|)$, polinomial em $n = |R| + |P|$.

Pseudo-código *first-fail*.

```
S, nao_cobertos <- {}, R
enquanto nao_cobertos != {}:
    r* <- argmin_{r em nao_cobertos} |G[r] \ S|
    p* <- argmax_{p em G[r*] \ S} |V[p] cap nao_cobertos|
    S <- S cup {p*}; nao_cobertos <- nao_cobertos \ V[p*]
```

2.2 Avaliação experimental

Para avaliar a qualidade das heurísticas, comparámo-las contra o ótimo obtido por Programação Inteira (modelo dos slides MAD2526_clp, *Postos de Vigia*, resolvido com Google OR-Tools / SCIP).

Geraram-se baterias de instâncias com o gerador `rectParts.c` (distribuição geométrica, $\tau = 0,75$) para $nr \in \{10, 20, 40, 80, 160\}$, com 20 instâncias por tamanho.

nr	ot. méd.	ff méd.	ff %ót.	ff exc. %	mc méd.	mc %ót.	mc exc. %	t_{PI} (ms)
10	3,80	4,00	80,0 %	6,25 %	4,15	65,0 %	10,00 %	4,6
20	7,20	7,85	50,0 %	9,29 %	8,55	0,0 %	18,93 %	5,5
40	14,40	16,20	0,0 %	12,65 %	17,15	5,0 %	19,27 %	13,5
80	27,60	31,50	0,0 %	14,14 %	33,60	0,0 %	21,78 %	24,4
160	55,25	63,05	0,0 %	14,15 %	68,80	0,0 %	24,54 %	102,0

Tabela 1: Q1: número médio de guardas (ót. = ótimo; ff = *first-fail*; mc = *max-cobertura*). “%ót.” = percentagem de instâncias em que a heurística atinge o ótimo. “exc.%” = excesso médio relativo ao ótimo.

Validação na fp3 ex. 7. Ambas as heurísticas atingem o ótimo conhecido ($|S| = 4$) na instância canónica, em parte porque essa instância contém retângulos com $|G[r]| = 1$, que forçam a escolha.

2.3 Discussão

***first-fail* domina *max-cobertura*.** Em todos os tamanhos a estratégia *first-fail* produz menos guardas. A diferença estabiliza em cerca de 10 pontos percentuais de excesso médio nas instâncias maiores ($\approx 14\%$ vs $\approx 24\%$ para $nr = 160$). Embora a *max-cobertura* seja a heurística clássica para *set cover*, com garantia de aproximação $H(|R|) = O(\log |R|)$, ela falha aqui porque ignora a estrutura geométrica do problema: ao maximizar a cobertura local pode deixar retângulos com poucos candidatos sem opções boas.

Estrutura das instâncias. Numa análise das instâncias geradas, verificou-se que praticamente nunca há retângulos com $|G[r]| = 1$ (i.e., guardas forçados), mas existe uma cauda de retângulos com $|G[r]| \in \{2, 3\}$. São estes que o *first-fail* ataca primeiro, capturando a maior parte do ganho heurístico.

Tempo. O tempo das heurísticas é desprezável (poucos ms); o solver exato continua rápido (102 ms em média para $nr = 160$). Para os tamanhos estudados não há vantagem em usar heurísticas em detrimento do ótimo; a sua utilidade reside na compreensão da estrutura do problema e como termo de comparação para variantes mais complexas (Q4).

3 Q2 — Programação Inteira e Programação por Restrições

3.1 Modelo Matemático

Para modelar o problema de vigilância de partições retangulares como um problema de otimização combinatória, partilhamos a mesma base teórica entre a Programação Inteira (PI) e a Programação por Restrições (CSP). Recorrendo ao pré-processamento estabelecido na Q1 (onde a borda externa é descartada), reutilizamos o conjunto de vértices candidatos válidos P e o conjunto de retângulos a cobrir R . O modelo define-se da seguinte forma:

- **Variáveis de decisão:** $x_p \in \{0, 1\}$ para cada $p \in P$, onde $x_p = 1$ indica a colocação de um guarda no vértice p .
- **Função objetivo:** minimizar o número total de guardas colocados na partição,

$$\min \sum_{p \in P} x_p.$$

- **Restrições de cobertura:** para garantir que todos os retângulos são vigiados, cada retângulo $r \in R$ tem de possuir pelo menos um guarda num dos seus vértices incidentes $G[r]$,

$$\sum_{p \in G[r]} x_p \geq 1 \quad \forall r \in R.$$

3.2 Implementação MAC (*Maintaining Arc Consistency*) e *Branch & Bound*

Na alínea (b) implementámos um algoritmo de pesquisa com retrocesso (*backtracking*) que não recorre a motores externos de CSP. Para garantir eficiência computacional, o algoritmo foi reforçado com duas técnicas cruciais:

- **Propagação de restrições (MAC):** em cada nó da árvore de pesquisa aplicamos consistência de arco. Se, após algumas atribuições, um retângulo ficar sem guardas atribuídos e com apenas um vértice livre restante no seu conjunto $G[r]$, o algoritmo força imediatamente a colocação de um guarda nesse vértice ($x_p = 1$). Se um retângulo ficar sem guardas e sem vértices livres, o ramo falha imediatamente.
- **Branch and bound (*pruning*):** o algoritmo mantém em memória o custo da melhor solução viável encontrada (S^*). Qualquer ramo da árvore de pesquisa que acumule um número de guardas igual ou superior a $|S^*|$ é imediatamente podado, reduzindo drasticamente o espaço de procura.

3.3 Soluções com motores externos: CP-SAT e Prolog (clpfd)

Para as alíneas seguintes, o mesmo modelo foi traduzido para ferramentas *state-of-the-art*:

- **Google OR-Tools (CP-SAT):** o modelo foi mapeado utilizando a interface `cp_model`, instanciando vetores de variáveis booleanas. As restrições foram aplicadas diretamente iterando sobre as listas de índices dos vértices incidentes a cada retângulo, tirando partido da inferência lógica do solver SAT subjacente.

- **SWI-Prolog (clpfd)**: a mesma lógica foi declarada em Prolog usando domínios finitos (`ins 0..1`). A cobertura de cada face foi garantida com a restrição `sum(VarsDaFace, #>=, 1)`, seguida de um predicado de otimização `labeling([min(MinGuardas)], Guardas)`.

4 Q3 — Programação Dinâmica

Para que a aplicação desta técnica valha a pena, devemos avaliar se é possível repartir o problema em subproblemas independentes e se isso compensa face às restantes soluções possíveis.

4.1 A dificuldade da subestrutura em grafos 2D arbitrários

Ao modelarmos a partição retangular num grafo (onde os retângulos são os elementos a cobrir e os vértices são os pontos de decisão), obtemos uma estrutura muito densa e com diversos ciclos.

A decisão de colocar um guarda num vértice v_i resolve a cobertura dos retângulos incidentes, mas altera o “estado” necessário para tomar decisões nos vértices vizinhos. Como o mapa é bidimensional, estas decisões propagam-se em várias direções, fechando ciclos e destruindo a independência local exigida pela Programação Dinâmica clássica (como a aplicada em árvores ou sequências lineares). O estado necessário para memorizar um subproblema não se resume a um único nó, mas sim a toda a fronteira de intersecção com o resto do mapa.

4.2 Programação Dinâmica de Perfil (*Profile DP*)

A única forma exata de aplicar Programação Dinâmica a este problema de grelha bidimensional seria através de uma técnica conhecida como Programação Dinâmica de Perfil (*Profile DP* ou varrimento de linha). O algoritmo funcionaria, em teoria, da seguinte forma:

1. Uma linha imaginária varre o mapa ortogonal da esquerda para a direita (ou de cima para baixo).
2. A linha intersesta, a qualquer momento, um conjunto de retângulos, formando uma “fronteira” ou “perfil” de largura W .
3. O estado da PD teria de memorizar todas as combinações possíveis de cobertura para os retângulos dessa fronteira.
4. Para cada nova coluna avançada, o algoritmo testaria todas as permutações válidas de guardas e faria a transição de estados.

4.3 Complexidade

A complexidade temporal e espacial da Programação Dinâmica de Perfil é governada pelo tamanho máximo da fronteira W . O número de estados a memorizar em cada passo cresce exponencialmente, na ordem de $\mathcal{O}(2^W)$. No nosso caso de estudo (por exemplo, uma instância com 40 retângulos e 82 vértices interligados num espaço denso), a largura do perfil W atinge valores que originam uma explosão combinatória abrupta na tabela de memorização. Conclui-se que, embora matematicamente possível, esta abordagem é computacionalmente impraticável e significativamente inferior às técnicas baseadas em restrições utilizadas na Q2.

5 Q4 — Extensões

5.1 Q4a — Guardas com cores

Enunciado e modelo. Cada guarda ativo recebe uma cor. Requisito: se dois guardas p e q veem ambos o mesmo retângulo r (i.e. $p, q \in G[r]$), as suas cores diferem. Em linguagem CSP/CLP é uma restrição `all_distinct` aplicada às cores dos guardas ativos em cada $G[r]$ — a

mesma primitiva usada nos slides MAD2526_clp (SEND+MORE, Sudoku) e em clp_swi_2026 (*quatroEstilos*).

Equivalentemente, define-se o *grafo de conflito* $H = (S, E)$ com $E = \{\{p, q\} : \exists r, p, q \in S \cap G[r]\}$; o número mínimo de cores é o número cromático $\chi(H)$.

Duas leituras da pergunta. A pergunta do enunciado — “para o número mínimo de guardas, qual seria o número mínimo de cores?” — admite duas interpretações, porque o número mínimo de guardas pode ser atingido por várias soluções diferentes, cada uma com um grafo de conflito distinto:

Leitura A (sequencial). Fixar uma solução ótima de guardas S e calcular $\chi(H_S)$ — o número mínimo de cores para *essa* colocação.

Leitura B (lexicográfica). Entre *todas* as colocações com o número mínimo de guardas G^* , escolher a que minimiza o número de cores.

Implementámos ambas. A Leitura B fornece uma resposta mais forte à pergunta; a Leitura A é a interpretação mais direta e útil quando a solução de guardas já está fixada por outras razões.

Modelo de Leitura B (lexicográfico). Variáveis booleanas $y_{p,k}$ = “guarda p ativo com cor k ” e usa_k = “cor k usada por algum guarda”.

$$\begin{aligned}
\min \quad & \sum_k usa_k \\
\text{s.a.} \quad & \sum_{p \in G[r]} x_p \geq 1, \quad r \in R \\
& \sum_p x_p = G^* \\
& \sum_k y_{p,k} = x_p, \quad p \in P \\
& \sum_{p \in G[r]} y_{p,k} \leq 1, \quad r \in R, k \\
& y_{p,k} \leq usa_k \\
& usa_k \geq usa_{k+1} \quad (\text{quebra de simetria}) \\
& x_p, y_{p,k}, usa_k \in \{0, 1\}
\end{aligned}$$

A restrição $\sum_{p \in G[r]} y_{p,k} \leq 1$ é exatamente a tradução para variáveis booleanas da restrição `all_distinct` sobre as cores dos guardas ativos em $G[r]$. Resolvido com CP-SAT (Google OR-Tools).

nr	guardas méd.	cores _A méd.	A máx.	cores _B méd.	B máx.	B prov. ót.
10	3,9	1,90	3	1,13	2	100 %
20	7,4	2,13	3	1,37	2	100 %
40	14,2	2,10	4	1,60	3	100 %
80	27,7	2,23	4	1,63	2	100 %

Tabela 2: Q4a: número de cores nas duas leituras.

Resultados experimentais (30 instâncias por tamanho).

Discussão.

- O número de cores mantém-se baixo em todas as instâncias estudadas (≤ 4 na Leitura A, ≤ 3 na Leitura B). A explicação geométrica é que o número de cores é limitado inferiormente pelo tamanho da maior *clique* de guardas que se veem todos no mesmo retângulo, e cada retângulo tem um número limitado de vértices candidatos.

- A Leitura B dá sistematicamente menos cores do que a Leitura A (média $\approx 1,5$ contra $\approx 2,1$). Em muitas instâncias basta uma cor na Leitura B, indicando que existe uma colocação ótima de guardas em que nenhuns dois guardas veem o mesmo retângulo.
- Resposta à pergunta do enunciado: para o número mínimo de guardas, o número mínimo de cores é tipicamente 1 ou 2 e raramente 3. Depende criticamente de qual solução ótima de guardas se escolhe — é o ponto subtil que distingue as duas leituras.

5.2 Q4b — Guardas com maior alcance

Enunciado e interpretação. “Permitir que um guarda num vértice possa guardar não só os retângulos incidentes mas também vizinhos a uma distância (no grafo) não superior a um valor D dado como parâmetro.”

A decisão central é a interpretação de “o grafo”. Adotámos a seguinte: *o grafo é o grafo dos vértices da subdivisão planar* — nós são os vértices (x, y) da partição; arestas são os segmentos entre vértices consecutivos ao longo das linhas da grelha. Justificação:

1. A frase do enunciado diz “um guarda *num vértice* ... distância (no grafo)” — a distância é naturalmente medida a partir do vértice onde o guarda está, no grafo onde esse vértice vive.
2. Para $D = 0$ (“vizinhos a distância 0” = o próprio vértice) recupera-se exatamente a definição base “vê as peças incidentes nesse vértice” (fp3 ex. 7). Verificámos isto computacionalmente.
3. O ficheiro `types.h` do gerador define adjacência entre vértices (campo `nxt[2]`: “próximo vértice na mesma linha XX e YY”). É a estrutura de grafo presente no código.

Modelo modificado. Idêntico ao modelo base, com $G[r]$ substituído por

$$G_D[r] = \{p \in P : \exists u, \text{dist}_{\text{grafo}}(p, u) \leq D \text{ e } u \in G[r]\}.$$

Algoritmo. O grafo não tem pesos, pelo que a distância mede-se em número de arestas. Calculamos $V_D[p]$ por *BFS limitado a profundidade D* (slide DAA2526_grafos5), caso particular de Dijkstra com pesos unitários (slide DAA2526_grafos3). O solver de cobertura é o mesmo da Q2a.

nr	$D = 0$	$D = 1$	$D = 2$	$D = 3$	$D = 4$
10	3,9	2,1	1,8	1,1	1,0
20	7,4	4,3	3,0	2,0	1,8
40	14,2	7,9	5,1	3,3	2,5
80	27,7	15,2	9,5	6,3	4,5

Tabela 3: Q4b: número médio de guardas em função do alcance D .

Resultados experimentais (30 instâncias por tamanho).

Discussão.

- O alcance reduz substancialmente o número de guardas necessários. Em $nr = 80$, passar de $D = 0$ para $D = 1$ corta o número de guardas quase para metade ($27,7 \rightarrow 15,2$). Cada incremento adicional continua a reduzir, com retornos decrescentes; para D suficientemente grande, um único guarda chega.
- Verificação: para $D = 0$ o número médio de guardas coincide exatamente com o valor reportado na Q1 (Tabela 1), confirmando que a extensão generaliza corretamente o problema base.

- Os tempos de resolução continuam baixos — o gargalo computacional é o solver de cobertura, não o BFS.

6 Conclusão

Resumo das contribuições:

- Na Q1, a estratégia *first-fail* sugerida pela alínea (a) do exercício 7 da fp3 traduz-se diretamente num algoritmo guloso eficaz, consistentemente superior à estratégia clássica de *max-cobertura*, com excesso médio estabilizado em $\approx 14\%$ relativamente ao ótimo nas instâncias maiores.
- Na Q2, a avaliação cruzada entre a implementação manual (MAC) e os motores externos (CP-SAT e Prolog) validou a exatidão do modelo matemático, retornando invariavelmente os mesmos custos ótimos. Destaca-se a observação empírica da simetria de soluções ótimas, em que diferentes solvers atingem o mesmo número mínimo de guardas, mas com alocações geográficas distintas devido às heurísticas de procura próprias de cada motor.
- Na Q3, a análise teórica e prática da Programação Dinâmica demonstrou que a densidade do grafo bidimensional planar e a imprevisibilidade da largura da fronteira (W) geram uma explosão combinatória exponencial ($\mathcal{O}(2^W)$). Conclui-se que esta abordagem, embora matematicamente possível, é computacionalmente impraticável e significativamente inferior às técnicas baseadas em restrições utilizadas na Q2.
- Na Q4a, distinguir as duas leituras da pergunta do enunciado revela um aspeto subtil: o número mínimo de cores depende de qual solução ótima de guardas se considera. Em ambas as leituras o número de cores é pequeno (≤ 4) e na Leitura B é tipicamente 1 ou 2.
- Na Q4b, com o grafo dos vértices como leitura natural de “o grafo”, o número de guardas decresce rapidamente com D e a generalização do modelo é direta.

A Estrutura da entrega

```
GRUPO_06/
relatorio.pdf          (este documento)
README.txt             (procedimentos de execucao)
programas/
  rectParts.c, types.h, macros.h  (gerador da professora)
  Q1/
    instance.py          -- leitura e pre-processamento
    greedy.py            -- estrategias first-fail e max-cobertura
    exato.py             -- solver de PI (OR-Tools, referencia otima)
    benchmark.py         -- estudo experimental da Q1
  Q2/
    instance_q2.py       -- leitura e pre-processamento
    ex2B_MAC_AC3.py     -- MAC (AC-3) com branch & bound
    ex2C_ORTools.py     -- solver CP-SAT (OR-Tools)
    ex2C_prolog.pl       -- solver clpfd (SWI-Prolog)
    ex2A.md              -- modelo matematico
  Q3/
    instance_q3.py       -- leitura e pre-processamento
    ex3_DP.py           -- programacao dinamica (bitmask)
    ex3.md              -- avaliacao da aplicabilidade da PD
  Q4/
    instance_q4.py       -- leitura com ordem e construcao do grafo
    alcance.py          -- Q4b (set cover com alcance D)
    cores.py            -- Q4a (Leituras A e B)
```

```
    bench_alcance.py  
    bench_cores.py  
casos_teste/  
    inst_10.txt, inst_20.txt, inst_40.txt, inst_80.txt
```